

Functional-style data processing with Java **Streams**

Marco Faella

Corso di Laurea in Informatica

Corso di Linguaggi di Programmazione II

Università di Napoli Federico II

A stream is...

...a sequence of objects...

...supporting **internal iteration**

I.e., the stream library takes *responsibility* for the iteration

List

- add
- remove
- search
- scan/iterate

Iterator

- scan/iterate
- (remove)

Stream

- internal iteration

Printing a sequence of objects

```
List<Integer> l = ...;  
for (Integer n: l)  
    System.out.println(n);
```

Printing a sequence of objects

```
List<Integer> l = ...;  
for (Integer n: l)  
    System.out.println(n);
```

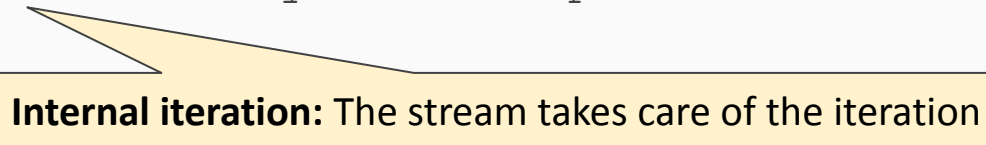
```
Iterator<Integer> i = ...;  
while (i.hasNext())  
    System.out.println(i.next());
```

Printing a sequence of objects

```
List<Integer> l = ...;  
for (Integer n: l)  
    System.out.println(n);
```

```
Iterator<Integer> i = ...;  
while (i.hasNext())  
    System.out.println(i.next());
```

```
Stream<Integer> s = ...;  
s.forEach(i -> System.out.println(i));
```



Internal iteration: The stream takes care of the iteration

Stream Interface

Interface `java.util.Stream<T>`

- 30+ instance methods
- 7 static methods

Internal Iteration

Repeatedly apply an operation to all elements

Responsibility shifts from client to stream library

foreach is a *terminal operation*

Types of Operations

Build ops:

create a stream from a data source

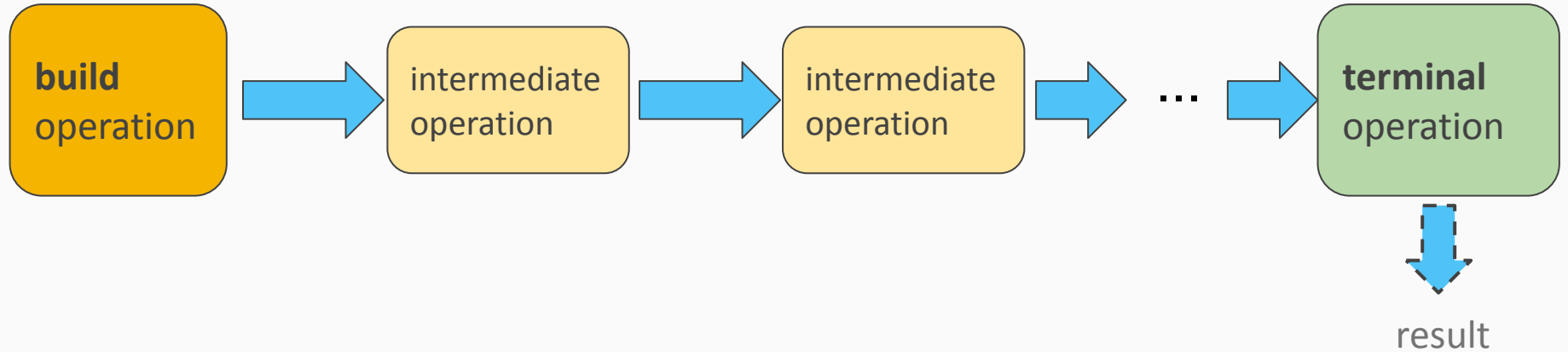
Intermediate ops:

convert one stream into another

Terminal ops:

convert stream into something else (or nothing)

A Pipeline of Stream Operations



Example

Print the names of the employees with salary at least 2500\$, alphabetically sorted

Example

Print the names of the employees with salary at least 2500\$, alphabetically sorted

```
Employee[] emps = { /* put employees here */ };  
  
Stream.of(emps).filter(e -> e.getSalary() >= 2500)  
    .map(e -> e.getName())  
    .sorted()  
    .forEachOrdered(  
        name -> System.out.println(name)) ;
```

Example

Build op

```
Stream.of(emps).filter(e -> e.getSalary() >= 2500)
           {
             .map(e -> e.getName())
             .sorted()
             .forEachOrdered(
               name -> System.out.println(name));
           }
```

Intermediate ops

Terminal op

Example, with Method References

Build op

```
Stream.of(emps).filter(e -> e.getSalary() >= 2500)
```

Intermediate ops

```
.map(Employee::getName)
```

```
.sorted()
```

Terminal op

```
.forEachOrdered(System.out::println);
```

Types of streams

Streams can be...

...ordered or unordered

...sequential or parallel

Ordered vs Unordered Streams

Objects in a stream may have a fixed order, or not
the *encounter order*

Ordered stream:

operations are performed in that order

Unordered stream:

operations may be performed in any order

Sequential vs Parallel Streams

Sequential stream:

operations are performed on one object at a time

Parallel stream:

operations may be performed on several objects in parallel

Creating Streams

Creating streams from...

...a static sequence of objects,

...a collection,

...a computation

Static sequences of objects

Stream from a static sequence of objects

Static method “**Stream.of**”

```
public static <T> Stream<T> of(T ... values)
```



variadic method

Example:

```
Stream<Integer> fib = Stream.of(1, 1, 2, 3);
```

Stream from a static sequence of objects

Static method “**Stream.of**”

```
public static <T> Stream<T> of(T ... values)
```

Example:

```
Stream<String> italianNumbers =  
    Stream.of("uno", "due", "tre");
```

Stream from an **array** of objects

Static method “**Stream.of**”

```
public static <T> Stream<T> of(T ... values)
```

Example:

```
Employee[] emps = ...  
Stream<Employee> empStream = Stream.of(emps);
```


Streams from *.of* are
ordered and *sequential*

Arrays and collections

Creating Streams from Arrays

In class **Arrays**:

```
public static <T> Stream<T> stream(T[] array)
public static <T> Stream<T> stream(T[] array,
                                   int startInclusive,
                                   int endExclusive)
```

Example:

```
Employee[] emps = ...
Stream<Employee> empStream = Arrays.stream(emps);
```

Creating Streams from Collections

In interface **Collection<T>**:

```
Stream<T> stream()  
Stream<T> parallelStream()
```

Example:

```
Collection<Employee> emps = ...  
Stream<Employee> empStream = emps.stream();
```

Streams from **lists** are *ordered*

Streams from **sets** are *unordered*

Non-interference

Streams must **not** *modify their source collection*

Similar rule for iterators

Streams from computations

Elements are computed **on demand**:

1. each one **separately** (via Supplier), or
2. each one **from the previous one** (via UnaryOperator)

Note: On demand computing is called “lazy evaluation”

Infinite Streams

Computed streams are (potentially) **infinite**

No bound on their length

Only a finite number of them will ever be created

Creating each Element Separately

In interface Stream:

```
static <T> Stream<T> generate (Supplier<T> s)
```

Generates an **unordered infinite** stream (don't worry, streams are *lazy*...)

Example (a stream of random integers):

```
Random random = new Random();  
Stream<Integer> randoms = Stream.generate(random::nextInt);
```

Creating each Element from the Previous One

In interface Stream:

```
static <T> Stream<T> iterate (T first,  
                               UnaryOperator<T> next)
```

Generates an **ordered infinite** stream

Example (a stream of sequences of a's):

```
Stream<String> as = Stream.iterate ("a", s -> s + "a");
```

Terminal Operations

Standard Terminations for Stream<T>

To **void**: forEach, forEachOrdered

To **long**: count

To **Collection<T>**: collect

Terminations to void

Execute a Consumer for each element

void **forEach** (Consumer<? super T> c)
 ignores the encounter order

void **forEachOrdered** (Consumer<? super T> c)
 respects the encounter order

Termination to long

Counts the number of elements

```
long count()
```

Termination into a Collection

Summarize using a **Collector**

On a `Stream<T> s`:

`s.collect(Collectors.toList())` returns `List<T>`

`s.collect(Collectors.toSet())` returns `Set<T>`

No guarantee on the actual type of List/Set

Note: The collect method is much more general...

Standard Terminations for Stream<T>

To **T**: findFirst, findAny, min, max

Terminations to An Element

Extract an element from the stream

`Optional<T>` **findAny**()
returns an **arbitrary** element

`Optional<T>` **findFirst**()
returns the **first** element in the encounter order

`Optional<T>` **min**(`Comparator<? super T> comp`)
returns the **minimum** element according to `comp`

`Optional<T>` **max**(`Comparator<? super T> comp`)
returns the **maximum** element according to `comp`

Optional Class

Functional-style alternative to returning “null”

Can be chained:

```
String result = stream.min(comparator).orElse("default");
```



Stream<String>

Note: It's a monad

Some Methods from Optional<T>

boolean **isPresent** ()
is there an object inside?

T **get** ()
unwraps the object, or throws *NoSuchElementException*

Plus:

flatMap (makes it a monad), map, filter, of (static factory)

Max Example

Objective: Find the longest string in a collection

```
Collection<String> strings = ...;
```

```
Optional<String> longest =
```

```
    strings.stream()
```

```
        .max(Comparator.comparingInt(String::length));
```

Stream Operations

Stream Operations

Instance methods of `Stream<T>`

Intermediate: return `Stream<T>` or `Stream<R>`

Terminal: return anything else, or *void*

Filtering Stream Elements

Stream Operations

Filtering Operations

Based on:

- content: `filter`
- amount: `limit`
- uniqueness: `distinct`

Filter Operation

Discards all elements which violate a Predicate

```
Stream<T> filter(Predicate<? super T> property)
```

```
interface Predicate<E> {  
    bool test(E elem);  
}
```

For a `Stream<Employee>`, it
accepts a `Predicate<Person>`

Limit Operation

Picks the first n elements

```
Stream<T> limit(long n)
```

Distinct Operation

Discards duplicates (according to *equals*)

```
Stream<T> distinct()
```

Example

Objective: Select 10 random positive distinct integers

Example

Objective: Select 10 random positive distinct integers

```
Random rand = new Random();  
Stream<Integer> randoms = Stream.generate(random::nextInt);  
  
randoms.filter(n -> n>0)  
      .distinct()  
      .limit(10)  
      .forEach(n -> System.out.println(n));
```

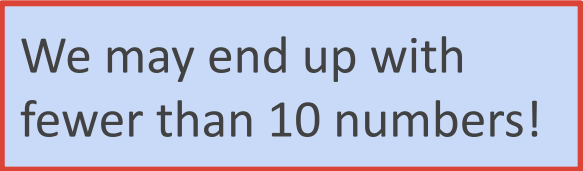
Note: Alternatively, the method `rand.ints()` returns a stream of primitive integers (`IntStream`)

Example

Objective: Select 10 random positive distinct integers

```
Random rand = new Random();  
Stream<Integer> randoms = Stream.generate(random::nextInt);
```

```
randoms.filter(n -> n>0)  
      .limit(10)  
      .distinct()  
      .forEach(System.out::println);
```



We may end up with
fewer than 10 numbers!

Example

Objective: Select 10 random positive distinct integers

```
Random rand = new Random();  
Stream<Integer> randoms = Stream.generate(random::nextInt);
```

```
randoms.distinct()  
    .filter(n -> n>0)  
    .limit(10)  
    .forEach(System.out::println);
```



It works,
but why spend time to remove
duplicate negative numbers?

About the Distinct Operation

One of the few *stateful* intermediate ops

It does *not* operate independently on each element

Harder to parallelize (see later...)

Transforming and Rearranging

Transforming Operations

Apply functions that modify content and type

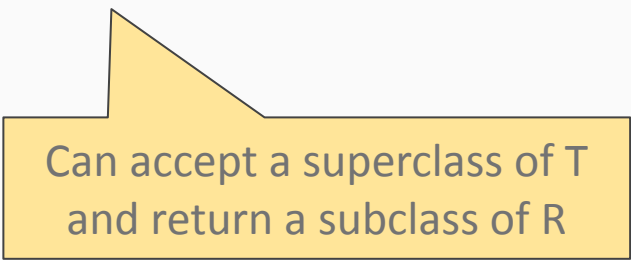
- function $T \rightarrow \text{Stream}\langle R \rangle$: **flatMap** (Section 4)
- function $T \rightarrow T$ or $T \rightarrow R$: **map**

Map Operation

Applies function to every element

In Stream<T>:

```
Stream<R> map(Function<? super T, ? extends R> fun)
```



Can accept a superclass of T
and return a subclass of R

Example

Assume **Employee** contains an **Address**, which contains a **City**

Objective: Print names of cities where at least one employee lives

Example

Assume **Employee** contains an **Address**, which contains a **City**

Objective: Print names of cities where at least one employee lives

```
Stream<Employee> emps = ...;
```

```
emps.map(Employee::getAddress)
```

Stream<Address>

```
.map(Address::getCity)
```

Stream<City>

```
.map(City::getName)
```

Stream<String>

```
.distinct()
```

```
.forEach(System.out::println);
```

Rearranging Operations

Sorting:

- based on natural order (*Comparable*)
- based on a *Comparator*

Unordering:

- convert stream to unordered
- release ordering guarantees

Sort Operations

According to a custom ordering

```
Stream<T> sorted()
```

```
Stream<T> sorted(Comparator<? super T> comp)
```

Stateful operations

Example

Objective: Print names of the 10 employees with the highest salary

```
Stream<Employee> emps = ...;
```

```
emps.sorted(  
    Comparator.comparingInt(Employee::getSalary)  
                .reversed()  
) .limit(10)  
  .map(Employee::getName)  
  .forEachOrdered(System.out::println);
```

Unordered Operation

Convert stream to unordered

```
Stream<T> unordered ()
```

- It does nothing to the data!
- Releases ordering guarantees (just a stream status flag)
- May improve parallel performance (Section 6)

Example

Objective: Efficiently count the number of distinct integers in a list

```
List<Integer> list = ...;
```

```
long n = list.parallelStream()  
             .unordered()  
             .distinct()  
             .count();
```

this stream is naturally ordered (from List)

relaxing the order gives **significant speedup**

Lazy Evaluation

Lazy = As Late As Possible (on demand)

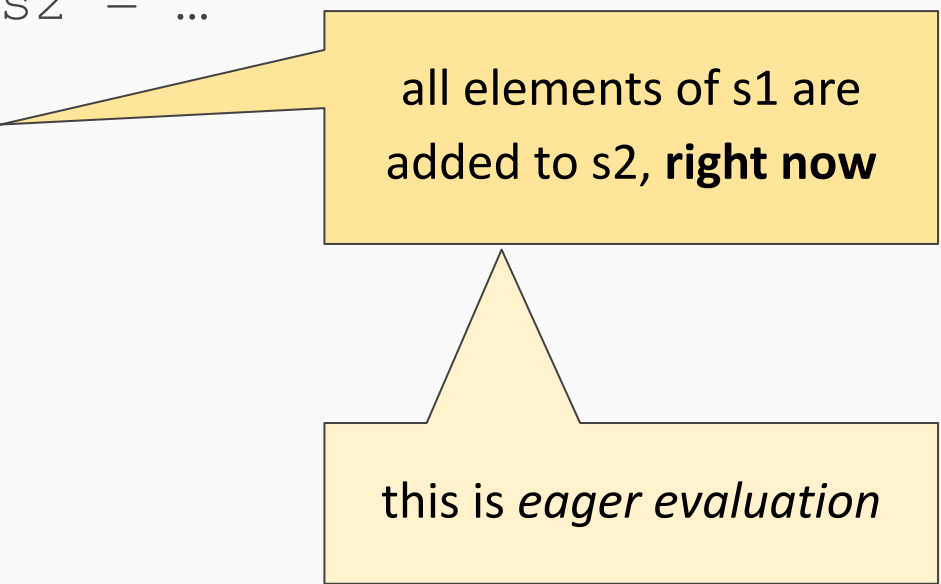
Eager = As Soon As Possible

An Eager Example

```
Set<Employee> s1 = ...
```

```
Set<Employee> s2 = ...
```

```
s2.addAll(s1);
```



all elements of s1 are
added to s2, **right now**

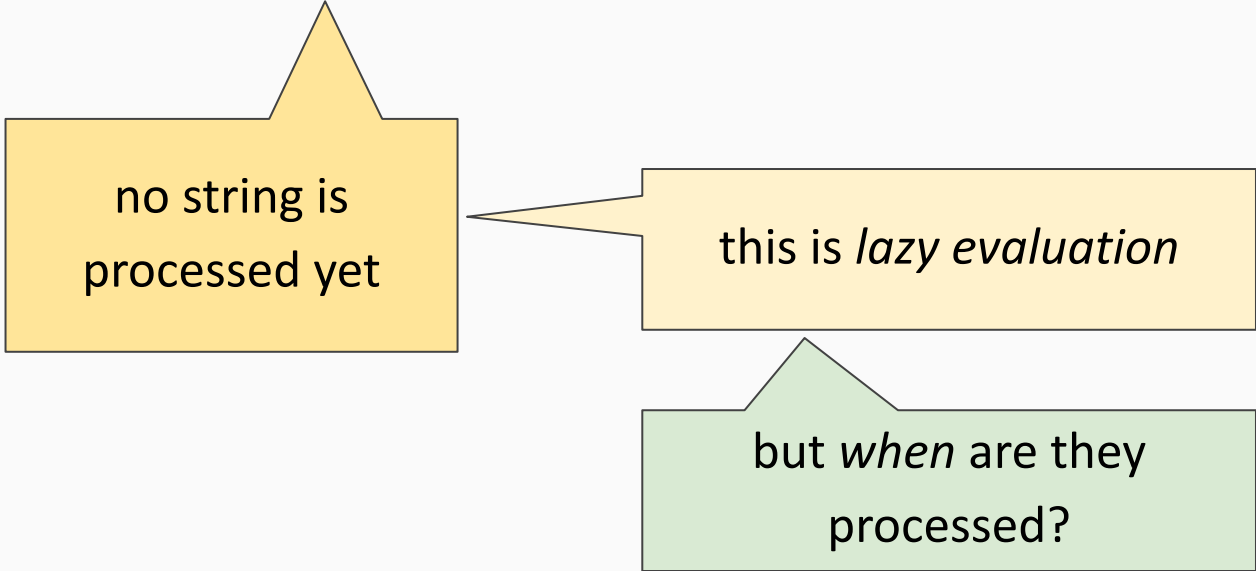
The diagram consists of a yellow callout box pointing to the `addAll` method call in the code above. Below this callout box is another yellow box containing the text "this is *eager evaluation*".

this is *eager evaluation*

A Lazy Example

```
List<String> content = ...
```

```
content.stream().distinct();
```



no string is
processed yet

The diagram consists of three callout boxes. A yellow box on the left points to the `stream()` method in the code above. A second yellow box to its right points to the `distinct()` method. A green box below the second yellow box points to the `distinct()` method as well.

this is *lazy evaluation*

but *when* are they
processed?

A Lazy Example

```
Random random = new Random();
```

```
Stream<Integer> randomness =  
    Stream.generate(random::nextInt);
```

no integer is
generated yet

this is *lazy evaluation*

but *when* are they
generated?

Elements are iterated and processed *when required by the terminal operation*

I.e.:

elements are *pulled from the end*

not *pushed from the start*

Example LazyTests.java

Streams can only be
traversed once

Traversing Twice

```
Stream<Integer> fib = Stream.of(1, 1, 2, 3);  
fib.forEach(System.out::println);  
fib.forEach(System.out::println);
```

→ 1 1 2 3

Exception in thread "main"

java.lang.**IllegalStateException**:

stream has already been operated upon or closed

Wrong Way to Split a Stream Pipeline

```
Stream<Integer> fib = Stream.of(1, 1, 2, 3);  
fib.limit(2);  
fib.forEach(System.out::println);
```

→ **Exception** in thread "main"

java.lang.IllegalStateException:

stream has already been operated upon or closed

Hint: intermediate ops return a *new stream*

Correct Way to Split a Stream Pipeline

```
Stream<Integer> fib = Stream.of(1, 1, 2, 3);  
Stream<Integer> shortFib = fib.limit(2);  
shortFib.forEach(System.out::println);
```

→ 1 1

Custom Reductions

Stream Operations

Custom Terminal Operations

Summarize a stream into a single object

Functional (i.e., *stateless*): **reduce**

Mutable (i.e., *stateful*): **collect**

Reduce: Functional summarization

A Common Summarization Pattern

Repeatedly apply a binary **operation**, starting from a *seed*

```
T summary = seed;  
for (T t: collection) {  
    summary = operation(summary, t);  
}
```

Examples:

- sum the elements
- compute the minimum/maximum

A Common Summarization Pattern

Repeatedly apply a binary **operation**, starting from a *seed*

```
T summary = seed;
for (T t: collection) {
    summary = operation(summary, t);
}
```

With streams:

```
T summary = collection.stream().reduce(seed, operation);
```

Reduce Terminal Operation

Summarize using a BinaryOperator

In `Stream<T>`:

```
T reduce(T seed, BinaryOperator<T> accumulator)
```

where:

```
interface BinaryOperator<T> {  
    T apply(T a, T b);  
}
```

Reduce Terminal Operation

Summarize using a BinaryOperator

```
T reduce(T seed, BinaryOperator<T> accumulator)
```

Example: string concatenation

```
Collection<String> words = ...  
String allWords = words.stream()  
    .reduce("", (a,b) -> a + " " + b);
```

Warning: Inefficient! Quadratic complexity! Use the builtin *joining* collector...

Exercise

Use “reduce” *without any other stream method*, to do the following:

- 1) Find the longest string in a `Collection<String>`
- 2) Compute the sum of the squares of all positive integers in a `Collection<Integer>`